

**OpenBank Cloud Simulation - IBM Full Stack Developer Edition**, now expanded with **clear weekly deliverables** that reflect measurable progress for both mentors and learners.

It's structured for use in your Coursera learning path - combining **IBM-aligned content**, **open-source tools**, and **realistic pacing**

## OpenBank Cloud Simulation - IBM Full Stack Developer Edition

**Duration:** 4 Weeks

**Learning Path Alignment:** IBM Full Stack Software Developer Professional Certificate

### Project Goal

Develop and deploy a **cloud-native full stack banking system** using **React**, **Node.js/FastAPI**, and **MongoDB**, deployed with **Docker** and **Kubernetes/OpenShift**. Learners will apply **full stack**, **DevOps**, and **cloud-native skills** in a simulated IBM Cloud environment using open-source tools.

### Learning Outcomes

By the end of the project, learners will:

- Build a **full stack web application** integrating frontend, backend, and database layers.
- Apply **cloud-native concepts** using containers, CI/CD, and orchestration.
- Implement **authentication**, **monitoring**, and **security best practices**.
- Document and present their project with a professional **GitHub portfolio**.

### Relevant Coursera Courses

| Course Title                                                                   | Focus Area                                    |
|--------------------------------------------------------------------------------|-----------------------------------------------|
| <a href="#">Introduction to Cloud Computing</a>                                | Cloud architecture and IBM Cloud foundations. |
| <a href="#">Introduction to Web Development with HTML, CSS, and JavaScript</a> | Frontend structure and responsive design.     |

| Course Title                                                  | Focus Area                                             |
|---------------------------------------------------------------|--------------------------------------------------------|
| <a href="#">React Basics</a>                                  | UI development and component-based architecture.       |
| <a href="#">Node.js and Express</a>                           | Backend API development and RESTful services.          |
| <a href="#">Python for Data Science, AI &amp; Development</a> | Optional Python scripting for backend or automation.   |
| <a href="#">Developing Cloud-Native Applications</a>          | Cloud-native design and microservices.                 |
| <a href="#">Containers &amp; Kubernetes Essentials</a>        | Container orchestration and deployment.                |
| <a href="#">Introduction to DevOps</a>                        | CI/CD automation and workflow integration.             |
| <a href="#">Application Security and Monitoring</a>           | Application monitoring, alerting, and dashboard setup. |

## 4-Week Realistic Sprint Plan

### Week 1 – Setup & Frontend Foundation

**Focus:** Environment setup, project structure, and user interface development.

#### Key Objectives

- Install required tools (Node.js, Docker, MongoDB, VS Code).
- Initialize GitHub repository and agile task board.
- Create the React frontend with basic routes (Login, Dashboard, Transactions).
- Design UI components and responsive layout.
- Understand cloud and web architecture concepts from IBM courses.

**Tools / Technologies:** React, HTML, CSS, Git, VS Code, Docker

**Relevant Courses:** *Intro to Cloud Computing, React Basics, Intro to Web Development*

## Week 1 Deliverables

- Functional **React UI** with pages for login, deposit, withdraw, and history.
- **GitHub repository** created and updated with README.
- **Project plan/board** on Trello or GitHub Projects.
- Screenshot of UI layout uploaded for peer/mentor feedback.

## Week 2 – Backend & Database Integration

**Focus:** API development, database connection, and authentication.

### Key Objectives

- Set up backend using Node.js (Express) or Python (FastAPI).
- Build RESTful APIs for user registration, deposit, withdrawal, and balance checks.
- Configure MongoDB with Mongoose/PyMongo.
- Implement JWT authentication and secure endpoints.
- Test endpoints with Postman.

**Tools / Technologies:** Node.js / FastAPI, MongoDB, JWT, Postman

**Relevant Courses:** *Node.js and Express, Application Security and Monitoring*

## Week 2 Deliverables

- Working **backend API** (POST/GET routes).
- **Database connected** with sample transaction data.
- **User authentication** with JWT (login token validated).
- Postman test results or screenshots in documentation.

## Week 3 – Cloud-Native Deployment & Automation

**Focus:** Containerization, orchestration, and CI/CD automation.

### Key Objectives

- Write Dockerfiles for frontend and backend.
- Create docker-compose.yml to run full stack locally.

## Key Objectives

- Set up GitHub Actions pipeline for build, test, and deploy automation.
- Deploy the system on Kubernetes (Minikube or OpenShift sandbox).
- Test cluster scaling and health.

**Tools / Technologies:** Docker, Docker Compose, GitHub Actions, Kubernetes, Minikube/OpenShift

**Relevant Courses:** *Developing Cloud-Native Applications, Containers & Kubernetes Essentials, Introduction to DevOps*

## Week 3 Deliverables

- **Dockerized frontend & backend** running locally.
- **CI/CD pipeline** triggers on Git push.
- **Deployment manifests** (Kubernetes YAML files).
- Screenshots or logs showing successful deployment to cluster.

## Week 4 – Monitoring, Optimization & Presentation

**Focus:** Observability, optimization, and professional documentation.

### Key Objectives

- Configure Prometheus for metrics collection.
- Build Grafana dashboards for API latency and transaction volume.
- Conduct testing and performance tuning.
- Prepare documentation and record a short video demo.
- Reflect on IBM-aligned skills gained.

**Tools / Technologies:** Prometheus, Grafana, GitHub Pages, Loom

**Relevant Courses:** *Application Security and Monitoring, Developing Cloud-Native Applications*

## Week 4 Deliverables

- **Monitoring dashboards** (Grafana) displaying live metrics.
- **Final project documentation** (README + diagrams).

- **Recorded video walkthrough/demo** (3–5 min).
- **GitHub link submission** with all code, configs, and notes.

### **Stretch Challenges (Optional for Advanced Learners)**

- Integrate **IBM Watson Assistant** for chatbot support.
- Add a **payment gateway simulation**.
- Deploy on **IBM Cloud Code Engine** instead of local Kubernetes.
- Add **email alerts** using RabbitMQ or MailHog (SNS equivalent).

### **Final Project Outcomes**

By the end of the 4-week sprint, learners will have:

- ✓ **A fully functional full stack banking system**
- ✓ Secure APIs and cloud-native deployment
- ✓ Automated CI/CD workflow
- ✓ Monitoring dashboards for observability
- ✓ **A portfolio-ready project** aligned with IBM's full stack learning path